Synthetic RGB-D of a manufacturing cell with a RealSense D435 depth model

Ravi Kola

November 2026

1 Overview

A humanoid robot in a manufacturing cell picks an angle bracket from a table next to an aluminium profile, using its head depth camera. We take the Unitree G1 as the robot and its head camera, an Intel RealSense D435, as the reference sensor. The D435 is an active infrared stereo camera, documented in Intel's datasheet [1] and depth tuning guide [2], and its stereo failure modes are the ones small metal features provoke. The work centres on the camera's depth model; the downstream task is deliberately simple.

We render labelled RGB-D frames of the cell in two simulators from one scene description, corrupt the ideal depth with an analytical D435 model built from published numbers, and run a depth-only pipeline on it: detect the bracket's top hole in 6D, fit the bracket's CAD model, and grasp the bracket with the jaws approaching along the hole axis. On 30 held-out test scenes, all of which succeed with ideal depth, the full camera model lowers hole detection from 100 % to 54 % and grasp success from 100 % to 57 %. The CAD fit still finds the bracket in 99 % of the cases, but displaced by about 11 mm. All code runs on a laptop.

2 Synthetic data pipeline

Scene. The parts are manufacturer CAD models from item (Line 8, 40×40): a profile cut to 200 mm and an angle bracket with its cover cap, simplified to free Ø8.5 mm holes. The table is sized to the G1's reach from an inverse-kinematics reachability map, which places the parts 0.47–0.68 m from the camera. Each random seed defines a scene: both parts lie in stable resting poses sampled by their probability, and the arms are raised to the sides so that the head camera sees the table unobstructed (Fig. 1).

Rendering and labels. A backend-independent scene description is rendered by PyBullet (CPU, about 2 s per frame) and by Isaac Sim 5.1 (RTX). The two agree on the parts' depth to 0.00 mm median, so the experiments use PyBullet, which reviewers can run on a laptop. Each frame holds an RGB image, ideal z-depth (848×480, the D435's recommended depth resolution, $f = 447$ px), an instance mask, and the camera intrinsics, camera pose and 6D part poses as ground truth. The camera model operates on saved frames and is therefore independent of the simulator. Building the pipeline exposed three rendering errors that would have biased every result: a half-pixel offset in PyBullet's image grid, Isaac ignoring a focal length given in stage units (rendering at 25° instead of 87° field of view), and the G1's waist sagging under soft joint drives so that the head camera moved while rendering.

3 D435 camera model

Approach. We corrupt the ideal depth with closed-form models, one effect at a time, so that each effect can be switched on alone and its downstream consequence measured. A physics-based simulation of the infrared stereo pair would be more faithful, but the D4 matcher's parameters are undocumented and the simulation would itself need validation against real data. Learned noise models need real D435 depth of similar scenes; we found no released D435 noise generator. The scope rule is that every modelled effect has a published number, so that the model can be traced to its sources and later calibrated.

Effects included. The D435 measures disparity d between its two infrared imagers, with baseline $B = 50$ mm, and reports depth $Z = fB/d$. Three effects are applied in the order in which they arise in the camera.

1. *Edge fattening.* A stereo matcher compares windows of pixels; at a depth step the window straddles both sides and the foreground wins, so the foreground grows into the background and narrow gaps close. No value is published for the D435. We use the only measured value, from Intel's previous R200 camera [3]: gaps narrower than 5 px merge, i.e. the foreground grows by 2 px.

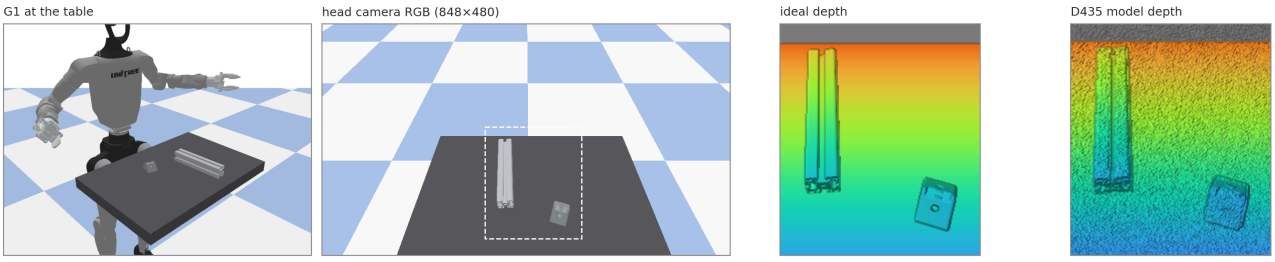


Figure 1: The cell and one frame (seed 204). From left: the G1 at the table in its observation pose, the head camera's RGB image, and the ideal and D435-model depth of the marked region.

2. *Axial noise*, added to the disparity of each pixel:

$$d' = d + s\,\xi, \quad \xi \sim \mathcal{N}(0,1) \qquad \Rightarrow \qquad \sigma_Z \approx \frac{Z^2}{fB}\,s, \tag{1}$$

   with Intel's subpixel accuracy $s = 0.08\,\text{px}$ [2], giving $\sigma_Z = 0.9\,\text{mm}$ at 0.5 m.
3. *Depth bias* at the datasheet's accuracy limit of ±2 %: $Z' = (1+b)\,Z$ with $b = \pm 0.02$, the sign drawn with equal probability. Since back-projection is linear in Z, every 3D point moves along its own viewing ray; shapes and orientations are preserved and the part shifts by about 10 mm.

The datasheet's output rules are applied last: the invalid band at the left image edge, the minimum depth of 195 mm and the 1 mm depth unit. Lateral error needs no separate term: in stereo the depth map lies on a fixed pixel grid, and lateral error arises from the axial noise through back-projection and from edge fattening at depth steps.

Effects left out. Several effects do not matter for a static, indoor, single-frame, depth-only task: temporal noise, rolling shutter and motion blur (the depth imagers have a global shutter), ambient infrared, interference between cameras, defocus, colour-sensor effects and post-processing filters (off by default). Others are absorbed by the pipeline: isolated invalid pixels fail the detector's size tests, and false depth on the profile's T-slots or specular dropouts on metal occur on the profile, which the CAD fit rejects; the bracket is matte powder-coated. For the remaining effects no usable number exists: the dependence on incidence angle (the only published term [4] is hand-set and does not reproduce its own measurements), occlusion shadows inside the image, and the spatially correlated "egg carton" noise that Intel describes without a correlation length.

The noise level itself is uncertain. Ahn et al. [4] fitted the axial noise of a real D435 to about 2.7 times Intel's value at 0.5 m, still within the range Intel measures on different surfaces (0.07–0.32 px). On the development scenes this level alone collapses the pipeline (CAD fit in 4 of 150 draws, against 143 with Intel's value), so the published noise level decides between a working and a failing perception stage.

4 Experiment: does the camera model change the downstream outcome?

Downstream task. The hole detector fits the table plane, peels planar faces off the remaining clusters with RANSAC, and accepts a hole as a round gap enclosed by a face; its centre lies on the face plane and its axis is the face normal. Its thresholds (1 mm to a plane, 1.5 mm above the table) were set on ideal depth and lie at the D435's noise level. The CAD fit starts from each upward hole candidate, searches the rotation about the hole axis and refines with ICP, keeping the best match. The grasp moves the Dex1 jaws along the hole axis, closes across the bracket and lifts it 10 cm; success is a lift above 50 mm with both jaws in contact. We chose this geometric pipeline over a learned pose estimator, which would need a GPU stack, because it relies on the same depth cues a network uses (faces, edges, holes a few pixels wide) and makes every failure traceable to its cause. A network trained on noisy depth may tolerate some effects, so the pipeline measures the sensitivity of the cues rather than the performance of a particular network.

Protocol. The pipeline, the camera model and the hypotheses were developed on 15 scenes, which were then closed. The test protocol was fixed before any test run (git tag `final-experiment`). The 30 test scenes are the first seeds from 201 on for which every stage succeeds with ideal depth, so that every failure is caused by the camera model. Seven depth conditions are compared: ideal, datasheet rules only, each effect alone, edge fattening with axial noise, and the full model. Each scene is run with 5 noise draws that are paired across conditions (same random numbers and bias sign), giving 150 draws per condition, compared with ideal by an

Condition	Hole detected	Pose found	Grasp lifted
ideal	100 %	100 %	100 %
datasheet rules	100 %	100 %	77 %
axial noise	56 %	99 %	63 %
edge fattening	100 %	100 %	63 %
edge + axial	39 %	100 %	65 %
depth bias ±2 %	100 %	100 % (10.7 mm off)	66 %
full model	54 %	99 % (10.9 mm off)	57 %

Table 1: Success per stage, 30 test scenes × 5 paired noise draws. Detection and pose within 15 mm.

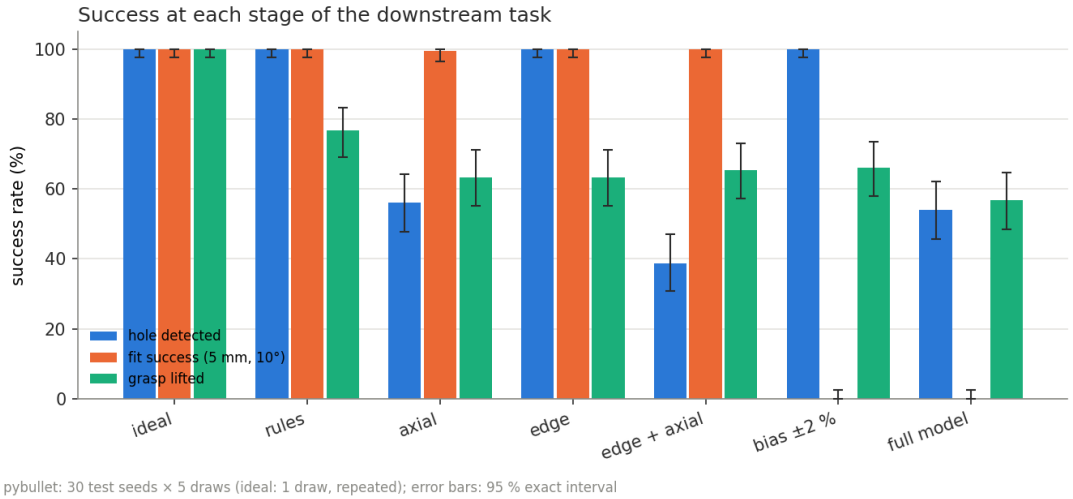


Figure 2: Success per stage and condition with 95 % exact intervals.

exact McNemar test. Because the bias moves every position by about 10 mm, detection and pose count as correct within 15 mm; the strict 3 mm and 5 mm criteria are reported as accuracy.

Results. Table 1 and Fig. 2 summarise the sweep. The datasheet rules leave perception unchanged (below 1 mm) but lower the grasp to 77 %: the 1 mm rounding nudges the fitted pose, and the grasp search switches to a candidate the arm cannot reach. This row is the control for the grasp planner's own sensitivity, against which the camera effects on the grasp should be read. Axial noise causes the largest single drop in detection, to 56 %: it leaves enclosed gaps on the profile's flat top face that pass as holes, and it breaks up the bracket's faces so that the real hole is no longer enclosed. Edge fattening alone does not prevent detection, since the plug it puts into the hole still stands out on noise-free depth, but it tilts the hole axis (0.75° to 1.12°) and with noise it lowers detection further, from 56 % to 39 % ($p = 0.005$, Fig. 3). The depth bias is invisible to detection but shifts the whole pose by 10.7 mm along the viewing ray and lowers the grasp to 66 %. The full model degrades every stage. Throughout, the CAD fit absorbs most perception errors, because it tries every candidate and keeps the one that matches the bracket.

What did not work. Three failure patterns remain. When axial noise breaks up every face, there is no hole candidate and the pipeline stops without a pose. When all candidates are false, the CAD fit can lock onto the profile (one case 253 mm and 43° off); the grasp planner then finds no reachable grasp, so the robot does not move. And near-correct poses still produce failed grasps (Fig. 4): the planner is sensitive to millimetre changes. The pipeline also has its own limits, which the test-scene selection hides. On scenes filtered only for a visible hole, ideal depth reaches 90 % detection (the misses pick a round gap in the bracket's cap) but only 47 % grasp success, mostly because the planner tries one grasp candidate and one hand orientation before reporting an IK failure.

Mitigation. Adapting the detector to the sensor recovers most of the loss. Replacing each threshold by $\max(\text{baseline}, k\,\sigma_Z(Z))$ with $k = 2.5$ raised detection on the development scenes from 60 % to 96 % and the CAD fit from 142 to 150 of 150, and 8 of 9 grasps that had started from a wrong hole then lifted the bracket. The accuracy loss stays (0.81 mm against 0.34 mm fit error with ideal depth), since it comes from the noisy points themselves, and wider thresholds tilt the hole axis. Tuned to Ahn's noise level, the detector reaches only

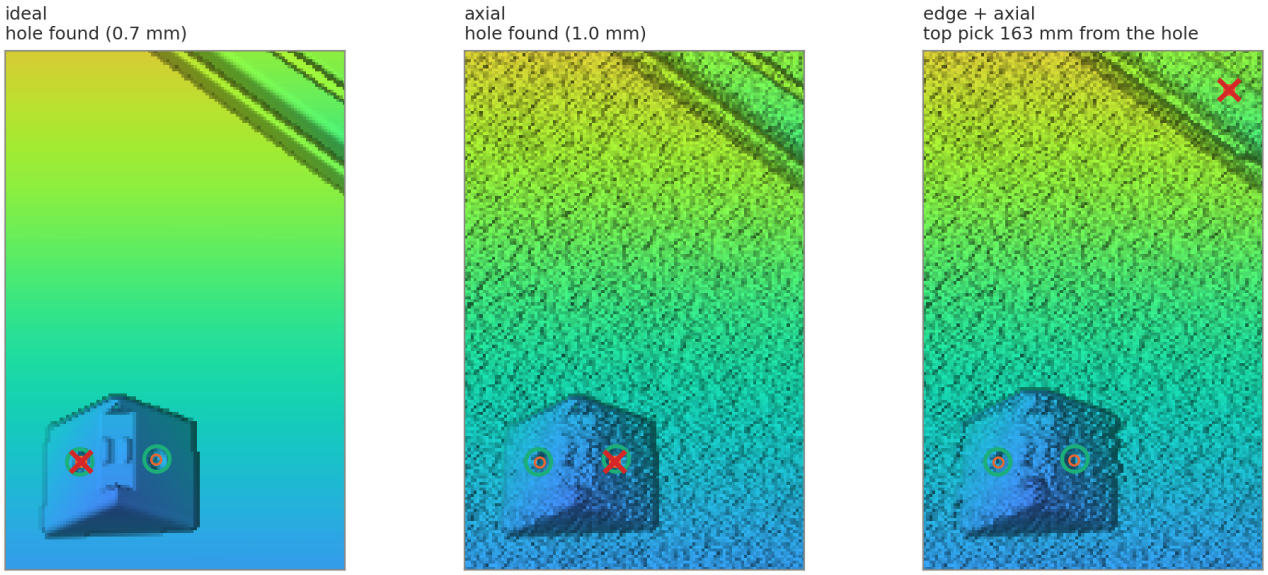


Figure 3: The same noise draw (seed 299) under three conditions. Green ring: true top hole; orange: hole candidates; red cross: the detector's pick. With axial noise the hole is still found; adding edge fattening fills it, and a false hole on the profile 163 mm away wins.

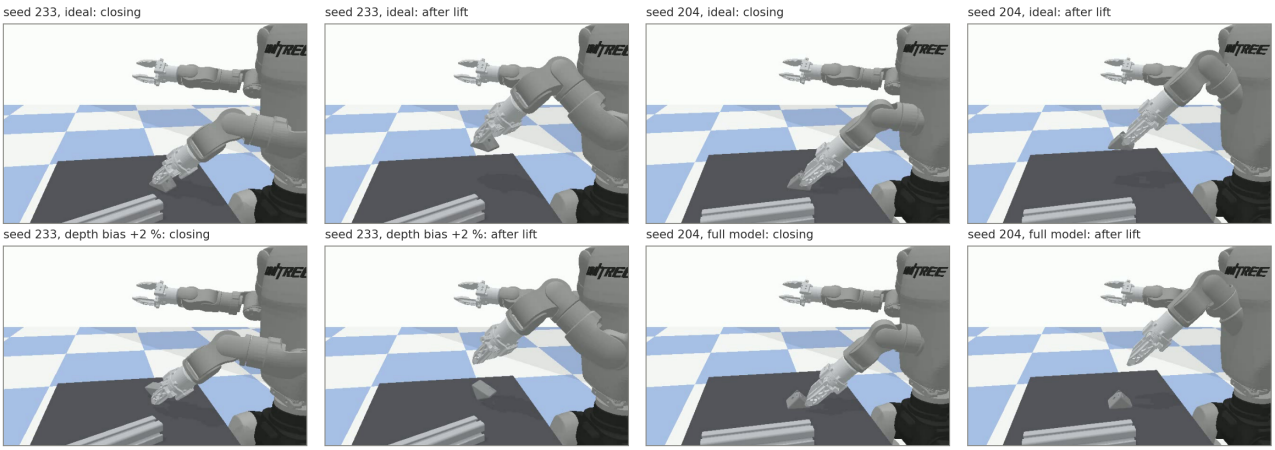


Figure 4: Grasps from the same scenes with ideal depth (top) and with the camera model (bottom). Left, seed 233: the depth bias places the grasp 10.7 mm off and the bracket is knocked over. Right, seed 204: under the full model the fit is 11.6 mm off and the jaws tip the bracket over.

36 %. This result is reported apart from the comparison, was tuned on the development scenes with the true noise level, and is not part of the test sweep.

Limitations. The evaluation is synthetic: it shows how sensitive the pipeline is to the model, not that the model is realistic. The edge-fattening width comes from the R200, the noise is independent per pixel, there is no angle dependence, and PyBullet renders no material effects. Validation against real D435 depth with ground truth (e.g. the HAMMER dataset) is the next step.

5 Riskiest sim-to-real assumption (draft)

A perception model adapts its parameters to the data it was developed on. The baseline detector, tuned on ideal depth, fails on D435 noise in the same way that a network or a robot policy trained on ideal synthetic depth would fail on the real camera. Generating the training data through a camera model adapts the parameters to the sensor that will be deployed, as the noise-aware detector shows: targeted data quality rather than the broad domain randomisation most sim-to-real approaches rely on, which brings its own difficulties for training.

The risk is that the adaptation is only as good as the camera model. The two published noise levels for the D435 differ by a factor of 2.7, and parameters adapted to the lower one meet the higher one as the ideal-tuned

detector met Intel's noise. The riskiest assumption is therefore that the simulated sensor matches the real one where it matters for the task.

First six months (draft). The work so far suggests measuring before modelling: validate the camera model against real D435 depth with ground truth, per depth range and separately at edges; calibrate the edge fattening on the D435 itself; and measure the angle dependence between 60° and 75°, where no data exist.

Use of AI tools

I used Claude Code throughout. It found, read and summarised the sources (datasheet, tuning guide, papers), looked up values and page references, wrote and ran the code under my decisions, and proofread the documents. I did not read the sources in full myself; instead I had claims checked against the source text, which found wrong page numbers, a citation attributed to the wrong camera (Keselman's values are for the R200) and a miscounted result. The main disagreement concerned edge fattening: the AI recommended modelling it with a swept width from the start and later ranked it first again. I declined, because no D435 value exists and an invented width would break traceability. In the end I tested it only with the one published number and marked it as needing calibration. The AI also recommended BlenderProc as the simulator; I chose Isaac Sim for its physics and robot integration and kept PyBullet so that the work runs on a laptop.

References

[1] Intel, *Intel RealSense D400 Series Datasheet*, rev. 017, 2026.

[2] A. Grunnet-Jepsen, J. N. Sweetser, J. Woodfill, *Best-Known-Methods for Tuning Intel RealSense D400 Depth Cameras for Best Performance*, Intel, rev. 2.0.

[3] L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, A. Bhowmik, Intel RealSense Stereoscopic Depth Cameras, *CVPR Workshops*, 2017.

[4] M. S. Ahn, H. Chae, D. Noh, H. Nam, D. Hong, Analysis and Noise Modeling of the Intel RealSense D435 for Mobile Robots, *Int. Conf. on Ubiquitous Robots (UR)*, 2019.